
boolean



JS - Array

Una variabile, tanti valori!

Array

Una variabile, tanti valori!

Se vogliamo salvare un **singolo valore** in una variabile, possiamo assegnarle un tipo di dato semplice (stringa, numero, booleano...).

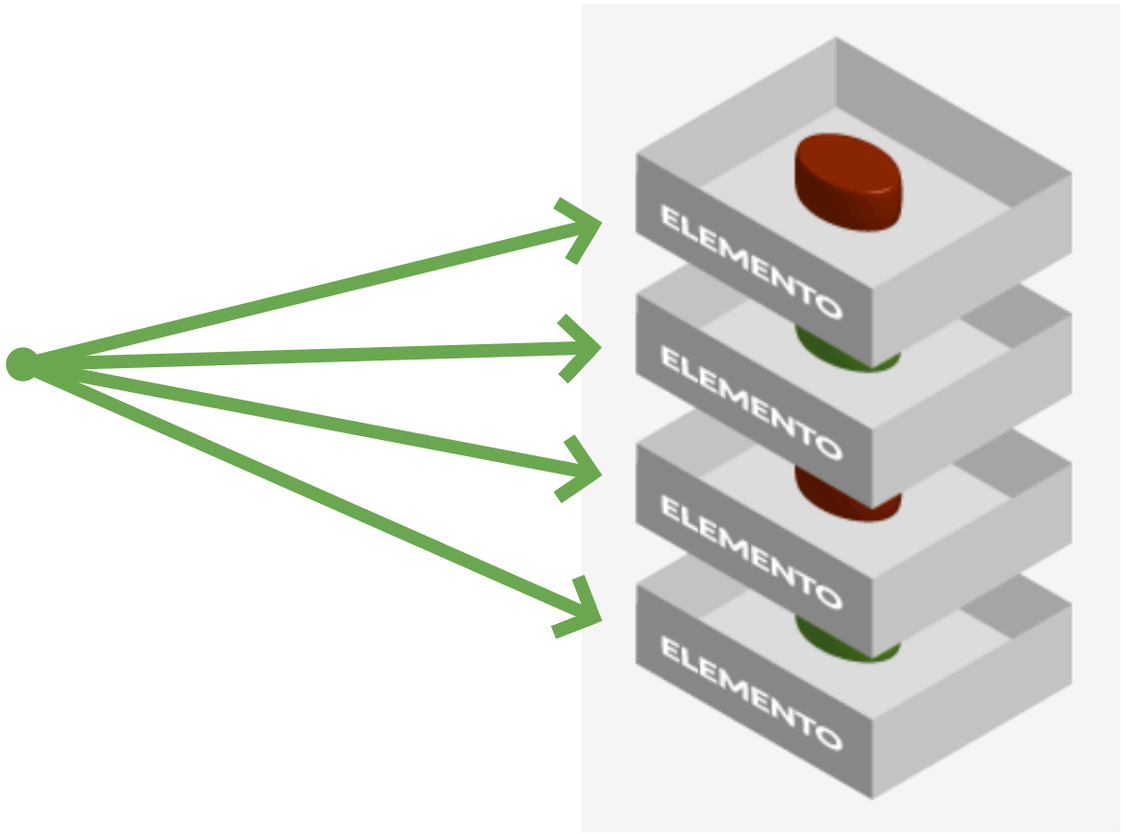
Ma se abbiamo bisogno di salvare **più valori collegati** in un'unica variabile?

Possiamo usare un **Array**!

Array

Una variabile, tanti valori!

🧐 L’**array** è come un archivio:
ogni **cassetto**, elemento, può contenere uno o più valori...

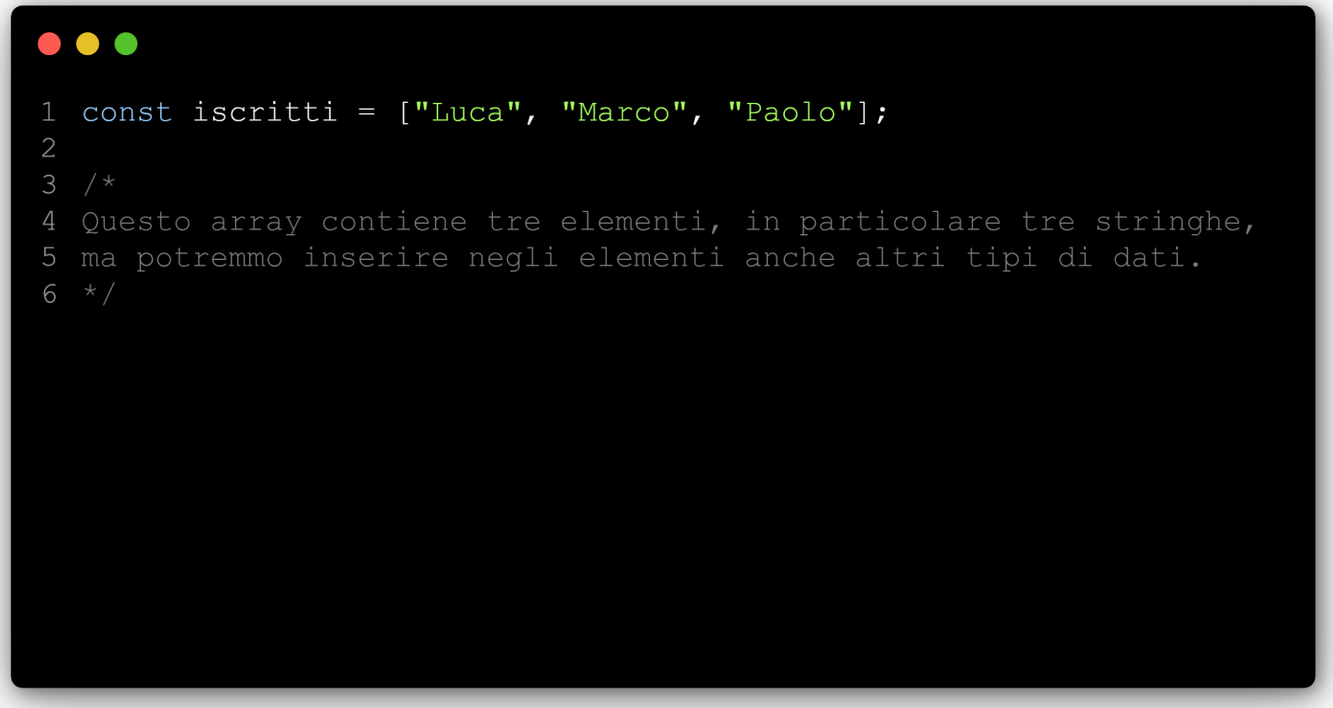


Array

Salviamo più dati assieme

Come si scrive un array?

Gli elementi di un array
si elencano tra parentesi quadre
e si separano con una virgola.



```
1 const iscritti = ["Luca", "Marco", "Paolo"];
2
3 /*
4 Questo array contiene tre elementi, in particolare tre stringhe,
5 ma potremmo inserire negli elementi anche altri tipi di dati.
6 */
```

Array

Salviamo più dati assieme

Come si accede agli elementi di un array?

nomeArray[IndiceElemento]

```
1 const iscritti = ["Luca", "Marco", "Paolo"];
2
3
4 iscritti[0] // Luca
5
6 iscritti[1] // Marco
7
8 iscritti[2] // Paolo
```

🧐 **Attenzione: gli array contano da 0!**

La prima **posizione/indice** quindi sarà 0 e non 1.

Array

E se voglio aggiungere un elemento?

Dopo la creazione di un array possiamo anche aggiungere degli altri dati.

Ad esempio qui gli iscritti iniziali sono 3, ma ad un certo punto potremmo aver bisogno di aggiungere un quarto elemento.

```
1 // creazione dell'array iniziale
2 const iscritti = ["Luca", "Marco", "Paolo"];
3
4 // aggiunta di un elemento all'array
5 iscritti.push('Michele');
6
```

```
// output dell'array modificato
["Luca", "Marco", "Paolo", "Michele"];
// .push() aggiunge un elemento alla fine
```

Array

Quindi posso modificare l'array anche se lo dichiaro con const?

Certamente! Quando dichiariamo una variabile come array, stiamo definendo il "contenitore" dei diversi valori, non i valori stessi.

E' come attaccare un'etichetta "giocattoli" su una scatola: il nome sull'etichetta serve per riferirsi alla scatola, non ai singoli giocattoli al suo interno.

Se quindi togliete o mettete giocattoli, l'etichetta si riferisce sempre alla stessa scatola, a prescindere dai giocattoli all'interno

```
1 // creazione dell'array iniziale
2 const giocattoli = ["Orsetto", "Trenino", ecc...];
3
4 // aggiunta di un elemento all'array
5 iscritti.push('Bambola');
6
7
8
```



Array

Quindi posso modificare l'array anche se lo dichiaro con const?

Questo naturalmente è possibile usando i metodi degli array.



```
// creazione dell'array iniziale
const giocattoli = ["Orsetto", "Trenino", ecc...];

// aggiunta di un elemento all'array
iscritti.push('Bambola');
```

Se invece provassimo a **riassegnare** un valore a una variabile otterremmo un errore!!



```
// creazione dell'array iniziale
const giocattoli = ["Orsetto", "Trenino", ecc...];

giocattoli = 'Altro';
```



Array + Cicli

❤️

Array + Cicli = ❤️

Abbiamo visto che per stampare gli elementi degli array possiamo usare il loro indice così:

```
1 const nomi = ["Luca", "Marco", "Paolo"];
2
3 //Stampa tutti i nomi
4 console.log(nomi[0]);
5
6 console.log(nomi[1]);
7
8 console.log(nomi[2]);
```



Ma se fossero 1000 nomi?

E se non fossero sempre 3 nomi, ma a volte 4 e a volte 2?

Array + Cicli = ❤️



Possiamo usare un ciclo for!

```
1 for (let i = 0; i < nomi.length; i++) {
2
3  console.log(nomi[i]);
4
5 }
```

.length è una proprietà degli array,
che ci dà la sua lunghezza,
quindi il numero di elementi contenuti

i, come abbiamo visto, avrà un valore
diverso ad ogni ciclo,
permettendoci di **attraversare** tutto l'array

Ciclo for con Array

Un esempio completo

1 **Creo il contatore**, una variabile che chiamo i, a cui do valore 0

Condizione: i è minore di 3?
Sì, allora **stampo nomi[0]** //Luca

2 **Incremento** il valore di i di 1 quindi ora i vale 1

Condizione: i è minore di 3?
Sì, allora **stampo nomi[1]** //Marco

3 **Incremento** il valore di i di 1 quindi ora i vale 2

Condizione: i è minore di 3?
Sì, allora **stampo nomi[2]** //Paolo

4 **Incremento** il valore di i di 2 quindi ora i vale 3

Condizione: i è minore di 3?
No, allora **esco dal ciclo**

```
1 const nomi = ["Luca", "Marco", "Paolo"];
2
3 for (let i = 0; i < nomi.length; i++) {
4
5  console.log(nomi[i]);
6
7 }
```

🧐 Gli array contano da ***0*** ma la loro **length** si conta da ***1***

nomi.length in questo caso è ***3***



LIVE CODING